

CS 446 Homework 2 NetID: anatoly2

1(a)(i)

Yes. By the chain rule of probability,

$$P(X, Y | Z) = P(X | Y, Z) P(Y | Z).$$

By the definition of conditional independence, $P(X | Y, Z) = P(X | Z)$. Substituting this into the equation above gives

$$P(X, Y | Z) = P(X | Z) P(Y | Z).$$

1(a)(ii)

No. Conditional independence given Z does not imply independence when Z is not observed. The following example shows this.

Let $Z \in \{0, 1\}$ with $P(Z = 1) = 0.5$. Given Z , let X and Y be independent binary variables with

$$\begin{aligned} P(X = 1 | Z = 1) &= P(Y = 1 | Z = 1) = 0.9, \\ P(X = 1 | Z = 0) &= P(Y = 1 | Z = 0) = 0.1. \end{aligned}$$

Then $X \perp Y | Z$ holds by construction. The marginal probabilities are

$$P(X = 1) = P(Y = 1) = 0.5 \cdot 0.9 + 0.5 \cdot 0.1 = 0.5.$$

The joint probability is

$$P(X = 1, Y = 1) = 0.5 \cdot 0.9^2 + 0.5 \cdot 0.1^2 = 0.41.$$

Since $0.41 \neq 0.25 = P(X = 1)P(Y = 1)$, the variables X and Y are not independent. In this example, X and Y both depend on Z . When Z is not observed, the value of X gives information about Z , and therefore also about Y .

1(a)(iii)

We need one parameter θ_{jc} for every attribute j and every class c . The probability $P(X_j = 0 | Y = c)$ is equal to $1 - \theta_{jc}$, so it does not need a separate parameter. There are d attributes and C classes. Therefore, dC independent parameters θ_{jc} must be estimated. The prior $P(Y = c)$ requires $C - 1$ additional parameters, but these are not θ_{jc} parameters.

1(a)(iv)

We need one mean μ_{jc} and one standard deviation σ_{jc} for every attribute j and every class c . Therefore, dC means and dC standard deviations must be estimated. This is $2dC$ parameters in total.

1(a)(v)

The denominator is

$$\sum_{v=1}^C P(Y = v) \prod_{k=1}^d P(X_k^{\text{new}} | Y = v) = P(X^{\text{new}}).$$

This sum runs over all classes v , so it does not depend on c . The denominator is the same positive number for every class c . Dividing all class scores by the same positive number does not change which score is the largest. Therefore, the $\arg \max$ does not change when the denominator is removed:

$$Y^{\text{new}} \leftarrow \arg \max_c P(Y = c) \prod_{j=1}^d P(X_j^{\text{new}} | Y = c).$$

1(a)(vi)

Yes. By the law of total probability and the Naive Bayes assumption,

$$P(X) = \sum_{c=1}^C P(Y = c) P(X | Y = c) = \sum_{c=1}^C P(Y = c) \prod_{j=1}^d P(X_j | Y = c).$$

Naive Bayes estimates all terms on the right-hand side. These terms are the priors $P(Y = c)$ and the class-conditional distributions of the attributes, for example θ_{jc} in the boolean case or μ_{jc} and σ_{jc} in the Gaussian case. This sum is the denominator in (v). The result is $P(X)$ under the Naive Bayes model. It is accurate only to the extent that the conditional independence assumption holds.

1(b)(i)

By the chain rule,

$$P(X_1, X_2, X_3, Y) = P(Y) P(X_1 | Y) P(X_2 | X_1, Y) P(X_3 | X_1, X_2, Y).$$

The problem states that $P(X_2 | X_1, Y) = P(X_2 | Y)$ and $P(X_3 | X_1, X_2, Y) = P(X_3 | X_1)$. Substituting these gives

$$P(X_1, X_2, X_3, Y) = P(Y) P(X_1 | Y) P(X_2 | Y) P(X_3 | X_1).$$

In this form, X_1 and X_2 depend only on Y , and X_3 depends only on X_1 .

1(b)(ii)

With the factorization from (i), the log-likelihood of the dataset is a sum of separate terms, one for each factor:

$$\log L = \sum_{i=1}^N \left[\log P(y^{(i)}) + \log P(x_1^{(i)} | y^{(i)}) + \log P(x_2^{(i)} | y^{(i)}) + \log P(x_3^{(i)} | x_1^{(i)}) \right].$$

Each factor can therefore be maximized separately. All variables are binary, so each factor is a Bernoulli distribution. The MLE of a Bernoulli parameter is the fraction of the relevant samples in which the event occurs. Therefore, each MLE below is a ratio of counts.

1(b)(ii)(A)

$P(Y)$ is one of the factors. Its MLE is the fraction of samples with label 1:

$$\hat{P}(Y = 1) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[y^{(i)} = 1].$$

1(b)(ii)(B)

$P(X_1 | Y)$ is one of the factors. Its MLE is the fraction of samples with $x_1 = 1$ among the samples with label y :

$$\hat{P}(X_1 = 1 | Y = y) = \frac{\sum_{i=1}^N \mathbb{1}[x_1^{(i)} = 1, y^{(i)} = y]}{\sum_{i=1}^N \mathbb{1}[y^{(i)} = y]}, \quad y \in \{0, 1\}.$$

1(b)(ii)(C)

$P(X_3 | Y)$ is not one of the factors. In the model, X_3 depends on Y only through X_1 . By the law of total probability over the two values $a \in \{0, 1\}$ of X_1 ,

$$P(X_3 = 1 | Y = y) = \sum_{a \in \{0, 1\}} P(X_3 = 1 | X_1 = a, Y = y) P(X_1 = a | Y = y).$$

In the model, X_3 depends only on X_1 , so $P(X_3 = 1 | X_1 = a, Y = y) = P(X_3 = 1 | X_1 = a)$. This gives

$$P(X_3 = 1 | Y = y) = \sum_{a \in \{0, 1\}} P(X_3 = 1 | X_1 = a) P(X_1 = a | Y = y).$$

Replacing each factor by its MLE gives

$$\begin{aligned} \hat{P}(X_3 = 1 | Y = y) &= \hat{P}(X_3 = 1 | X_1 = 1) \hat{P}(X_1 = 1 | Y = y) \\ &\quad + \hat{P}(X_3 = 1 | X_1 = 0) (1 - \hat{P}(X_1 = 1 | Y = y)), \end{aligned}$$

where $\hat{P}(X_1 = 1 \mid Y = y)$ is given in (B) and

$$\hat{P}(X_3 = 1 \mid X_1 = a) = \frac{\sum_{i=1}^N \mathbb{1}[x_3^{(i)} = 1, x_1^{(i)} = a]}{\sum_{i=1}^N \mathbb{1}[x_1^{(i)} = a]}, \quad a \in \{0, 1\}.$$

All denominators are nonzero because every conditioning event occurs at least once in the dataset. This estimate is in general different from the direct count

$$\frac{\sum_{i=1}^N \mathbb{1}[x_3^{(i)} = 1, y^{(i)} = y]}{\sum_{i=1}^N \mathbb{1}[y^{(i)} = y]},$$

because the fitted model allows X_3 to depend on Y only through X_1 .

1(c)(i)

The MAP estimate is the label with the largest posterior probability. By Bayes' rule and the factorization from (b)(i),

$$\begin{aligned} \hat{y} &= \arg \max_y P(Y = y \mid x_1, x_2, x_3) \\ &= \arg \max_y P(Y = y) P(x_1 \mid y) P(x_2 \mid y) P(x_3 \mid x_1). \end{aligned}$$

The prior is 0.5 for both labels, so it does not affect the $\arg \max$. The term $P(x_3 \mid x_1) = \mathcal{N}(x_3; 2x_1, 1)$ does not depend on y , so it does not affect the $\arg \max$ either. The remaining terms are Gaussian densities with variance 1. Their normalizing constants are the same for both labels. Therefore,

$$\begin{aligned} \hat{y} &= \arg \max_y \exp \left(-\frac{1}{2} [(x_1 - \mu_{1y})^2 + (x_2 - \mu_{2y})^2] \right) \\ &= \arg \min_y [(x_1 - \mu_{1y})^2 + (x_2 - \mu_{2y})^2]. \end{aligned}$$

For $y = 0$, the expression is $x_1^2 + x_2^2$. For $y = 1$, the expression is $(x_1 - 1)^2 + (x_2 - 1)^2 = x_1^2 + x_2^2 - 2(x_1 + x_2) + 2$. The value for $y = 1$ is smaller when $x_1 + x_2 > 1$. Therefore,

$$\hat{y} = \begin{cases} 1, & \text{if } x_1 + x_2 > 1, \\ 0, & \text{if } x_1 + x_2 < 1. \end{cases}$$

If $x_1 + x_2 = 1$, either label can be chosen. The rule assigns the label whose mean, $(0, 0)$ or $(1, 1)$, is closer to (x_1, x_2) . The rule does not use x_3 .

1(c)(ii)

For both points, $x_1 + x_2 = 0.2 + 0.7 = 0.9 < 1$. Therefore, $\hat{y} = 0$ for both $X^{(a)}$ and $X^{(b)}$. As a check, the expression for $y = 0$ is $0.2^2 + 0.7^2 = 0.53$, and the expression for $y = 1$ is $0.8^2 + 0.3^2 = 0.73$. The value for $y = 0$ is smaller.

The predicted labels do not differ. The two points differ only in x_3 . In this model, x_3 depends on Y only through X_1 . Since x_1 is observed, x_3 gives no additional information about y . The term $P(x_3 | x_1)$ has the same value for $y = 0$ and $y = 1$, so it cancels in the arg max. Both $x_3 = -10$ and $x_3 = 10$ have a very low density under $\mathcal{N}(0.4, 1)$, but this density is the same for both labels. Therefore, x_3 does not change the prediction.

2(d)

The MAP rule predicts $\hat{y} = 1$ when $P(Y = 1 | \mathbf{x}) > P(Y = 0 | \mathbf{x})$. By Bayes' rule, and since $P(\mathbf{x})$ is the same on both sides, this condition is equivalent to

$$\pi \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma) > (1 - \pi) \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_0, \Sigma).$$

The Gaussian density has the form $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = c \cdot \exp(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}))$, where the constant $c > 0$ depends only on Σ and d . Both classes have the same Σ , so c is the same on both sides and cancels. Taking the logarithm of both sides gives

$$\begin{aligned} \log \pi - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_1) \\ > \log(1 - \pi) - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_0)^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}_0). \end{aligned}$$

Since Σ^{-1} is symmetric, $\mathbf{x}^\top \Sigma^{-1} \boldsymbol{\mu} = \boldsymbol{\mu}^\top \Sigma^{-1} \mathbf{x}$. Therefore, each quadratic form expands as

$$(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu}) = \mathbf{x}^\top \Sigma^{-1} \mathbf{x} - 2 \boldsymbol{\mu}^\top \Sigma^{-1} \mathbf{x} + \boldsymbol{\mu}^\top \Sigma^{-1} \boldsymbol{\mu}.$$

The term $\mathbf{x}^\top \Sigma^{-1} \mathbf{x}$ appears on both sides of the inequality, so it cancels. This cancellation uses the assumption that both classes have the same covariance matrix. The inequality becomes

$$\begin{aligned} \log \pi + \boldsymbol{\mu}_1^\top \Sigma^{-1} \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_1^\top \Sigma^{-1} \boldsymbol{\mu}_1 \\ > \log(1 - \pi) + \boldsymbol{\mu}_0^\top \Sigma^{-1} \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_0^\top \Sigma^{-1} \boldsymbol{\mu}_0. \end{aligned}$$

Moving the terms that contain $\mathbf{x}$ to the left side and the other terms to the right side gives

$$(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \Sigma^{-1} \mathbf{x} > \frac{1}{2}(\boldsymbol{\mu}_1^\top \Sigma^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \Sigma^{-1} \boldsymbol{\mu}_0) + \log \frac{1 - \pi}{\pi}.$$

Define

$$\mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0), \quad \tau = \frac{1}{2}(\boldsymbol{\mu}_1^\top \Sigma^{-1} \boldsymbol{\mu}_1 - \boldsymbol{\mu}_0^\top \Sigma^{-1} \boldsymbol{\mu}_0) + \log \frac{1 - \pi}{\pi}.$$

Since Σ^{-1} is symmetric, $\mathbf{w}^\top \mathbf{x} = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \Sigma^{-1} \mathbf{x}$. The MAP rule is therefore

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & \text{if } \mathbf{w}^\top \mathbf{x} > \tau, \\ 0, & \text{if } \mathbf{w}^\top \mathbf{x} < \tau. \end{cases}$$

This is a linear classifier in $\mathbf{x}$. The decision boundary is the hyperplane $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} = \tau\}$. If the two classes had different covariance matrices, the terms $\mathbf{x}^\top \Sigma_y^{-1} \mathbf{x}$ would not cancel, and the decision boundary would be quadratic.

3(a)

Multiplying the numerator and the denominator by e^{-s} gives

$$\sigma(-s) = \frac{1}{1 + e^s} = \frac{e^{-s}}{e^{-s} + 1} = \frac{(1 + e^{-s}) - 1}{1 + e^{-s}} = 1 - \frac{1}{1 + e^{-s}} = 1 - \sigma(s).$$

The model is $P(y^{(i)} | \mathbf{x}^{(i)}) = \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)})$. Using this property,

$$\begin{aligned} P(y^{(i)} = -1 | \mathbf{x}^{(i)}) + P(y^{(i)} = 1 | \mathbf{x}^{(i)}) &= \sigma(-\mathbf{w}^\top \mathbf{x}^{(i)}) + \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}) \\ &= 1 - \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}) + \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}) = 1. \end{aligned}$$

3(b)

We write $\sigma(s) = (1 + e^{-s})^{-1}$ and apply the chain rule:

$$\sigma'(s) = -(1 + e^{-s})^{-2} \cdot (-e^{-s}) = \frac{e^{-s}}{(1 + e^{-s})^2} = \frac{1}{1 + e^{-s}} \cdot \frac{e^{-s}}{1 + e^{-s}} = \sigma(s)(1 - \sigma(s)).$$

The last step uses $\frac{e^{-s}}{1 + e^{-s}} = 1 - \sigma(s)$, which was shown in (a).

3(c)

The samples are independent. Therefore, the likelihood is a product over the samples, and the log-likelihood is a sum:

$$\ell(\mathbf{w}) = \log P(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \sum_{i=1}^N \log \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)}).$$

By (b), the derivative of $\log \sigma(s)$ is $\frac{\sigma'(s)}{\sigma(s)} = 1 - \sigma(s)$. Let $s_i = y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)}$. Then $\nabla_{\mathbf{w}} s_i = y^{(i)} \mathbf{x}^{(i)}$. By the chain rule,

$$\nabla_{\mathbf{w}} \log P(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \sum_{i=1}^N (1 - \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)})) y^{(i)} \mathbf{x}^{(i)}.$$

Each term is the vector $y^{(i)} \mathbf{x}^{(i)}$ multiplied by the weight $1 - \sigma(s_i)$. This weight is close to 1 when example i is misclassified by a large margin. It is close to 0 when example i is classified correctly by a large margin.

3(d)

From (c), the a -th entry of the gradient is

$$\frac{\partial \ell}{\partial w_a} = \sum_{i=1}^N (1 - \sigma(s_i)) y^{(i)} x_a^{(i)},$$

where $s_i = y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)}$ and $x_a^{(i)}$ is the a -th entry of $\mathbf{x}^{(i)}$. We differentiate this expression with respect to w_b . By (b), the derivative of $1 - \sigma(s_i)$ with respect to s_i is $-\sigma(s_i)(1 - \sigma(s_i))$. Also, $\frac{\partial s_i}{\partial w_b} = y^{(i)} x_b^{(i)}$. Therefore,

$$H_{ab} = \frac{\partial^2 \ell}{\partial w_a \partial w_b} = - \sum_{i=1}^N \sigma(s_i)(1 - \sigma(s_i)) (y^{(i)})^2 x_a^{(i)} x_b^{(i)}.$$

Since $y^{(i)} \in \{-1, +1\}$, we have $(y^{(i)})^2 = 1$. Therefore,

$$H_{ab} = - \sum_{i=1}^N \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)}) (1 - \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)})) x_a^{(i)} x_b^{(i)}.$$

In matrix form, the Hessian is

$$\mathbf{H} = - \sum_{i=1}^N \sigma(s_i)(1 - \sigma(s_i)) \mathbf{x}^{(i)} \mathbf{x}^{(i)\top}.$$

By (a), $\sigma(s)(1 - \sigma(s)) = \sigma(-s)(1 - \sigma(-s))$. Therefore, the factor $\sigma(s_i)(1 - \sigma(s_i))$ does not depend on the sign of $y^{(i)}$, and it is equal to $\sigma(\mathbf{w}^\top \mathbf{x}^{(i)})(1 - \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}))$.

3(e)

Let $\mathbf{z} \in \mathbb{R}^{d+1}$. We use the matrix form from (d) and the identity $\mathbf{z}^\top \mathbf{x}^{(i)} \mathbf{x}^{(i)\top} \mathbf{z} = (\mathbf{z}^\top \mathbf{x}^{(i)})^2$:

$$\mathbf{z}^\top \mathbf{H} \mathbf{z} = - \sum_{i=1}^N \sigma(s_i)(1 - \sigma(s_i)) (\mathbf{z}^\top \mathbf{x}^{(i)})^2.$$

For every i , $0 < \sigma(s_i) < 1$, so $\sigma(s_i)(1 - \sigma(s_i)) > 0$. Also, $(\mathbf{z}^\top \mathbf{x}^{(i)})^2 \geq 0$. Therefore, every term of the sum is nonnegative. Because of the minus sign, $\mathbf{z}^\top \mathbf{H} \mathbf{z} \leq 0$ for every $\mathbf{z}$. Therefore, $\mathbf{H}$ is negative semi-definite. This implies that the log-likelihood is concave, and any local maximum, if one exists, is a global maximum.

3(f)(i)

For every finite s , $0 < \sigma(s) < 1$, so $\log \sigma(s) < 0$. The training set is nonempty, so $N \geq 1$. Therefore, for every finite $\mathbf{w}$,

$$\ell(\mathbf{w}) = \sum_{i=1}^N \log \sigma(y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)}) < 0.$$

Now let $\mathbf{w} = t\mathbf{v}$ and $m_i = y^{(i)} \mathbf{v}^\top \mathbf{x}^{(i)}$. By linear separability, $m_i > 0$ for every i . Then

$$\ell(t\mathbf{v}) = \sum_{i=1}^N \log \sigma(t m_i).$$

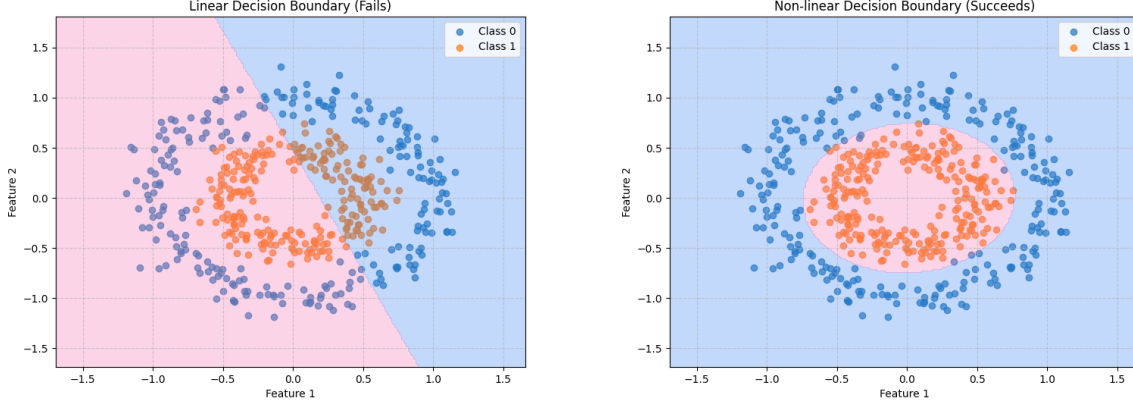
As $t \rightarrow \infty$, $t m_i \rightarrow \infty$ for every i , because $m_i > 0$. Therefore, $\sigma(t m_i) \rightarrow 1$ and $\log \sigma(t m_i) \rightarrow 0$ for every i . The sum has a finite number of terms. Therefore, $\lim_{t \rightarrow \infty} \ell(t\mathbf{v}) = 0$.

3(f)(ii)

Assume that a finite vector $\mathbf{w}^*$ maximizes ℓ . By (i), $\ell(\mathbf{w}^*) < 0$. Also by (i), $\ell(t\mathbf{v}) \rightarrow 0$ as $t \rightarrow \infty$. Therefore, there is a value of t with $\ell(t\mathbf{v}) > \ell(\mathbf{w}^*)$. This contradicts the assumption that $\mathbf{w}^*$ is a maximizer. Therefore, no finite $\mathbf{w}$ maximizes the log-likelihood. The supremum of ℓ is 0, and ℓ does not reach this value at any finite $\mathbf{w}$.

This result does not contradict (e). Concavity implies that a local maximum, if one exists, is a global maximum. Concavity does not imply that a maximum exists. A concave function can increase toward a limit without reaching it. For example, $f(t) = \log \sigma(t) = -\log(1+e^{-t})$ is concave and increasing, and $f(t) \rightarrow 0$ as $t \rightarrow \infty$, but f has no maximum. The log-likelihood behaves in the same way along the direction $\mathbf{v}$.

4(a)(iii)



The left plot shows the decision boundary of the linear model. The boundary is a straight line that cuts through both circles. The linear model has a training accuracy of 54.6% and a test accuracy of 46.7%. Its final cost is 0.691. This value is close to $\log 2 \approx 0.693$, which is the cost of predicting 0.5 for every point. The right plot shows the decision boundary of the model with the transformed features. The boundary is an ellipse around the inner circle. This model has a training accuracy of 98.6% and a test accuracy of 97.3%.

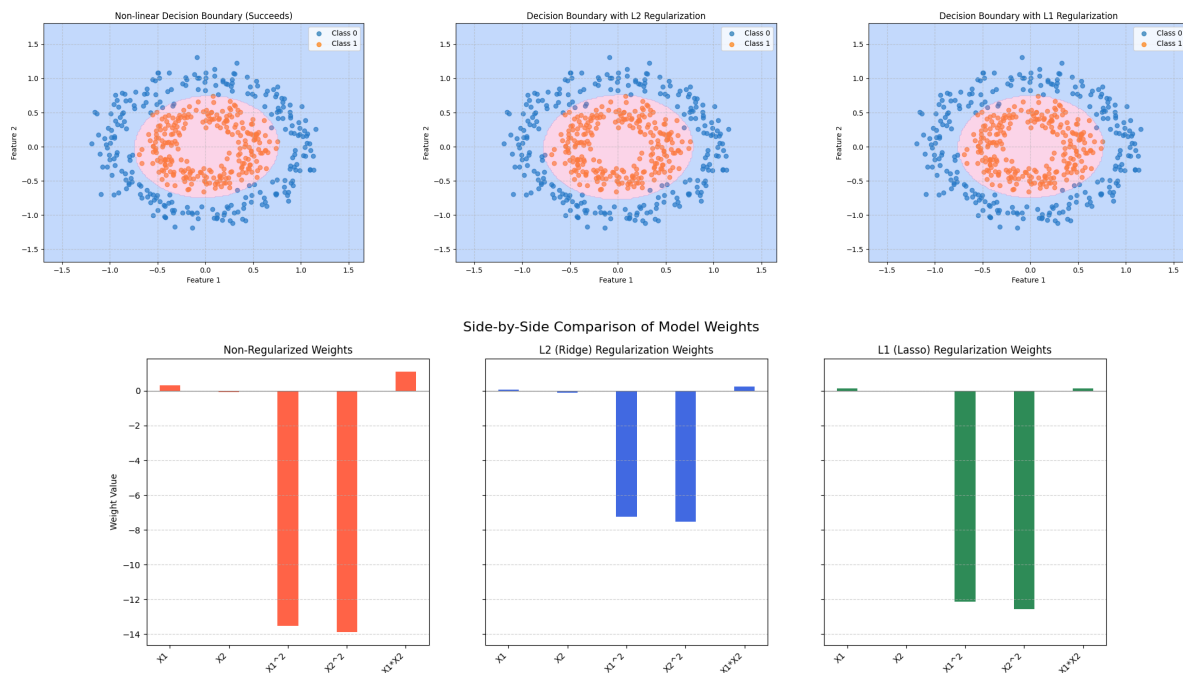
In the original feature space, logistic regression predicts class 1 when $w_1x_1 + w_2x_2 + b > 0$. The decision boundary $w_1x_1 + w_2x_2 + b = 0$ is a straight line. In this dataset, class 1 forms the inner circle, and class 0 forms a ring around it. A straight line cannot separate the inner circle from the ring. Therefore, the linear model cannot separate the two classes.

With the transformed features, the model is linear in the five features x_1, x_2, x_1^2, x_2^2 and x_1x_2 . Its decision boundary is

$$w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + b = 0.$$

In the original plane, this equation describes a quadratic curve, for example an ellipse. The learned weights of x_1^2 and x_2^2 are -13.53 and -13.88 , and the bias is 7.70 . The weights of x_1, x_2 and x_1x_2 are $0.33, -0.08$ and 1.10 , which are much smaller. If the three small terms are ignored, the model predicts class 1 when $13.53x_1^2 + 13.88x_2^2 < 7.70$. This region is the inside of an ellipse with semi-axes of about 0.75 . The ellipse lies between the inner circle, which has radius 0.5 , and the outer circle, which has radius 1 . Therefore, the two classes cannot be separated by a linear rule in the original feature space, but they can be separated by a linear rule in the transformed feature space, apart from a few noisy points.

4(b)(ii)



The first row of plots shows the decision boundaries of the model without regularization, the model with L2 regularization, and the model with L1 regularization, from left to right. The two regularized models use $\lambda = 0.5$. The bar chart shows the weights of the three models.

The three decision boundaries are similar ellipses. The training accuracies are 98.6% without regularization, 98.0% with L2 regularization, and 98.3% with L1 regularization. The test accuracies are 97.3%, 98.7% and 98.7%. The difference in test accuracy is one point out of 75 test points, so it is too small to draw a conclusion from it.

Regularization adds a penalty on the size of the weights to the cost. During training, the model has to balance a good fit to the training data against small weights. As a result, regularization makes the weights smaller. Smaller weights give a simpler model that depends less on individual training points, and this can reduce overfitting. Smaller weights also make the predicted probabilities change more slowly near the decision boundary. The sum of the absolute values of the weights is 28.9 without regularization, 15.2 with L2 regularization, and 25.0 with L1 regularization.

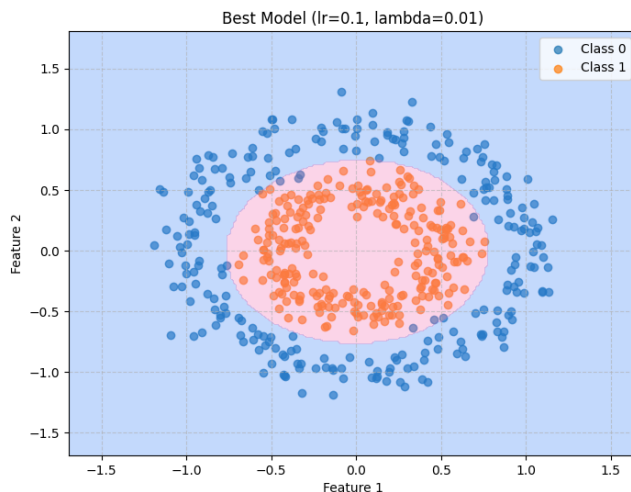
In the code, the L2 penalty is $\frac{\lambda}{2m} \sum_j w_j^2$, and its gradient is $\frac{\lambda}{m} w_j$. This gradient is proportional to the weight. Therefore, the L2 penalty pushes large weights toward zero much more strongly than small weights. In the bar chart, the

weights of x_1^2 and x_2^2 change from -13.53 and -13.88 to -7.24 and -7.54 . The small weights stay small, and no weight becomes zero.

The L1 penalty is $\frac{\lambda}{m} \sum_j |w_j|$, and its subgradient is $\frac{\lambda}{m} \text{sign}(w_j)$. This subgradient has the same size for every nonzero weight. Therefore, L1 regularization pushes small weights to zero and has a relatively small effect on large weights. In the bar chart, the weight of x_2 becomes -0.0002 , which is close to zero. The weight of $x_1 x_2$ changes from 1.10 to 0.16 . The weights of x_1^2 and x_2^2 change only to -12.14 and -12.57 .

The main difference is that L1 regularization produces sparse weights, in which some weights are zero or close to zero. L2 regularization makes all weights smaller but does not set them to zero. L1 regularization is a good choice when we expect that only some of the features are relevant and we want the model to select them. L2 regularization is a good choice when we expect that most features are relevant, or when the features are correlated, and we want to shrink all weights smoothly. L2 regularization is also differentiable everywhere, which makes it easier to optimize.

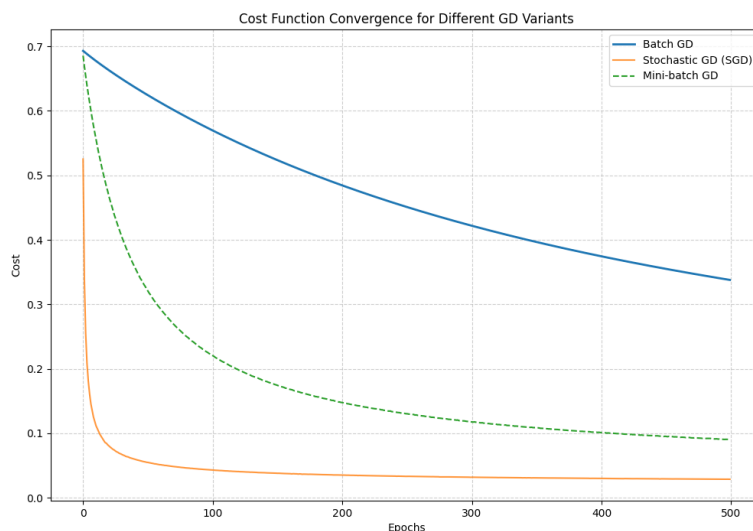
4(c)(ii)



The grid search uses the learning rates $\alpha \in \{0.01, 0.1, 1.0\}$ and the regularization strengths $\lambda \in \{0.01, 0.1, 1.0\}$. Each model uses L2 regularization and 5000 iterations. With $\alpha = 0.1$ and with $\alpha = 1.0$, the validation accuracy is 100% for all three values of λ . With $\alpha = 0.01$, the validation accuracy is 98.67% for all three values of λ . For $\lambda = 0.01$, the training cost after 5000 iterations is 0.338 with $\alpha = 0.01$, 0.097 with $\alpha = 0.1$, and 0.048 with $\alpha = 1.0$. The smallest learning rate takes the smallest steps, so after 5000 iterations its model is further from the minimum.

The code keeps the first combination that reaches the highest validation accuracy. Therefore, the selected combination is $\alpha = 0.1$ and $\lambda = 0.01$. The validation set has 75 points, and it does not distinguish between the six combinations that reach 100%. The final model is trained on the training set of 350 points with $\alpha = 0.1$ and $\lambda = 0.01$. Its test accuracy is 98.67%. The plot shows its decision boundary, which is an ellipse around the inner circle.

4(d)(ii)



The plot shows the average cost in each epoch for batch gradient descent, stochastic gradient descent, and mini-batch gradient descent with batch size 32. All three methods use the learning rate 0.1 and run for 500 epochs on the 350 training points.

- Batch gradient descent makes 1 update per epoch. Its cost decreases from 0.693 to 0.338 over 500 epochs, and it decreases in every epoch. Each update uses the exact gradient over the full training set, so the convergence is the most stable. However, this method makes the least progress per epoch.
- Mini-batch gradient descent makes 11 updates per epoch. Its cost decreases from 0.686 to 0.090 over 500 epochs.
- Stochastic gradient descent makes 350 updates per epoch. Its cost decreases from 0.525 after the first epoch to 0.118 after 10 epochs and to 0.029 after 500 epochs.

In this run, the methods that make more updates per epoch make more progress per epoch.

The gradient of a mini-batch or of a single sample is a noisy estimate of the full gradient. Therefore, the cost of stochastic and mini-batch gradient descent does not decrease in every step. The plotted value is an average over all updates in one epoch, and this averaging hides most of the noise. The plotted curves still increase in some epochs. This happens in 102 of the 499 changes from

one epoch to the next for stochastic gradient descent and in 67 of them for mini-batch gradient descent. Each of these increases is smaller than 0.0005.

However, the methods that make more progress per epoch also take more time per epoch. In this run, 500 epochs took 0.02 s with batch gradient descent, 0.12 s with mini-batch gradient descent, and 3.3 s with stochastic gradient descent. Batch gradient descent performs one vectorized computation per epoch. Stochastic gradient descent runs the Python loop 350 times per epoch, and each iteration does a small computation.

Batch gradient descent has the most stable convergence and the shortest time per epoch, but it needs the most epochs. Stochastic gradient descent needs the fewest epochs, but each epoch takes the longest time, and its updates are the noisiest. Mini-batch gradient descent is between the two methods in progress per epoch, time per epoch, and stability.